

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology
Practical - 27

Roll No.:S046	Name: Abhishek Yadav
Class: BSC IT	Batch:-----
Date of Assignment:22/9/2026	Date/Time of Submission:22/9/2026

Aim: Implement one-hot encoding for a categorical column in a dataset.

Code:

```
build.sbt  Demo27.scala x
1  object Demo27 extends App
2  {
3    val data = List("Red", "Blue", "Green", "Red", "Blue")
4    for (value <- data)
5    {
6      val r = if (value == "Red") 1 else 0
7      val b = if (value == "Blue") 1 else 0
8      val g = if (value == "Green") 1 else 0
9      println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)
10   }
11   println("\nAbhishek_S046")
12 }
```

Output:

```
Run  Demo27 x
C:\Program Files\Java\jdk1.8.0_341\bin\java.exe" ...
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0
Green -> Red:0 Blue:0 Green:1
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0

Abhishek_S046
```

Practical- 28

Aim: Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification.

Code:

```
build.sbt  Demo28.scala x
1  import breeze.linalg._
2
3  object Demo28 extends App
4  {
5      val A = DenseVector(2.0, 3.0)
6      val B = DenseVector(5.0, 7.0)
7
8      val distance = norm(A - B)
9
10     println("Euclidean Distance: " + distance)
11
12     val C = DenseVector(2.5, 3.5)
13
14     val d1 = norm(C - A)
15     val d2 = norm(C - B)
16
17     if (d1 < d2)
18     | println("Nearest Neighbor: A")
19     else
20     | println("Nearest Neighbor: B")
21
22     println("\nAbhishek_S046")
23 }
```

Output:

```
"C:\Program Files\Java\jdk1.8.0_341\bin\java.exe" ...
Euclidean Distance: 5.0
Nearest Neighbor: A
Abhishek_S046
Process finished with exit code 0
```

Practical- 29

Aim: Create a simple Spark MLlib pipeline to classify data. Use logistic regression or decision trees.

Code:

```
build.sbt × Demo29.scala
1 ThisBuild / version := "0.1.0-SNAPSHOT"
2
3 ThisBuild / scalaVersion := "2.13.12"
4
5 libraryDependencies += Seq(
6   "org.apache.spark" %% "spark-sql" % "3.5.9",
7   "org.apache.spark" %% "spark-mllib" % "3.5.9"
8 )
9
10 lazy val root = (project in file("."))
11   .settings(
12     name := "Demo29"
13   )
```

build.sbt Demo29.scala x

```
1 import org.apache.spark.sql.SparkSession
2 import org.apache.spark.ml.feature.VectorAssembler
3 import org.apache.spark.ml.classification.LogisticRegression
4
5 object Demo29 extends App {
6
7     val spark = SparkSession.builder()
8         .appName("Classification")
9         .master("local[*]")
10        .getOrCreate()
11
12    import spark.implicits._
13
14    val data = Seq(
15        (1.0, 0.0),
16        (2.0, 0.0),
17        (3.0, 0.0),
18        (7.0, 1.0),
19        (8.0, 1.0),
20        (9.0, 1.0)
21    ).toDF("marks", "label")
22
23    val a = new VectorAssembler()
24        .setInputCols(Array("marks"))
25        .setOutputCol("features")
26
27    val d = a.transform(data)
```

```

28
29     val model = new LogisticRegression()
30         .setLabelCol("label")
31         .setFeaturesCol("features")
32
33     val result = model.fit(d)
34
35     val prediction = result.transform(d)
36
37     prediction.select("marks", "label", "prediction").show()
38
39     spark.stop()
40 }
41

```

Output:

```

+-----+-----+-----+
|marks|label|prediction|
+-----+-----+-----+
|  1.0|  0.0|        0.0|
|  2.0|  0.0|        0.0|
|  3.0|  0.0|        0.0|
|  7.0|  1.0|        1.0|
|  8.0|  1.0|        1.0|
|  9.0|  1.0|        1.0|
+-----+-----+-----+

```